

Garbage & Earthquake

Garbage Collection Robot

Scenario

Design an autonomous Garbage Collection Robot for a smart city environment. The robot should patrol roads, parks, and public areas, detect garbage using vision and distance sensors, collect the waste using a robotic arm, classify it into recyclable and non-recyclable categories, and deposit it into separate bins. The robot should avoid obstacles, navigate safely, and send collection status to a monitoring station through Wi-Fi.

1. Description

The Garbage Collection Robot is an embedded robotic system designed to automate waste management in urban areas. The robot continuously moves through a predefined route while monitoring its surroundings using a camera and ultrasonic sensors. When garbage is detected, the robot approaches the object, identifies its type using image processing, and picks it up using a robotic arm equipped with a gripper.

The collected waste is then sorted into recyclable or non-recyclable compartments. If obstacles such as pedestrians, vehicles, or walls are encountered, the robot adjusts its path to avoid collisions. The robot can also transmit operational data such as bin capacity, battery level, and collection statistics to a remote monitoring system.

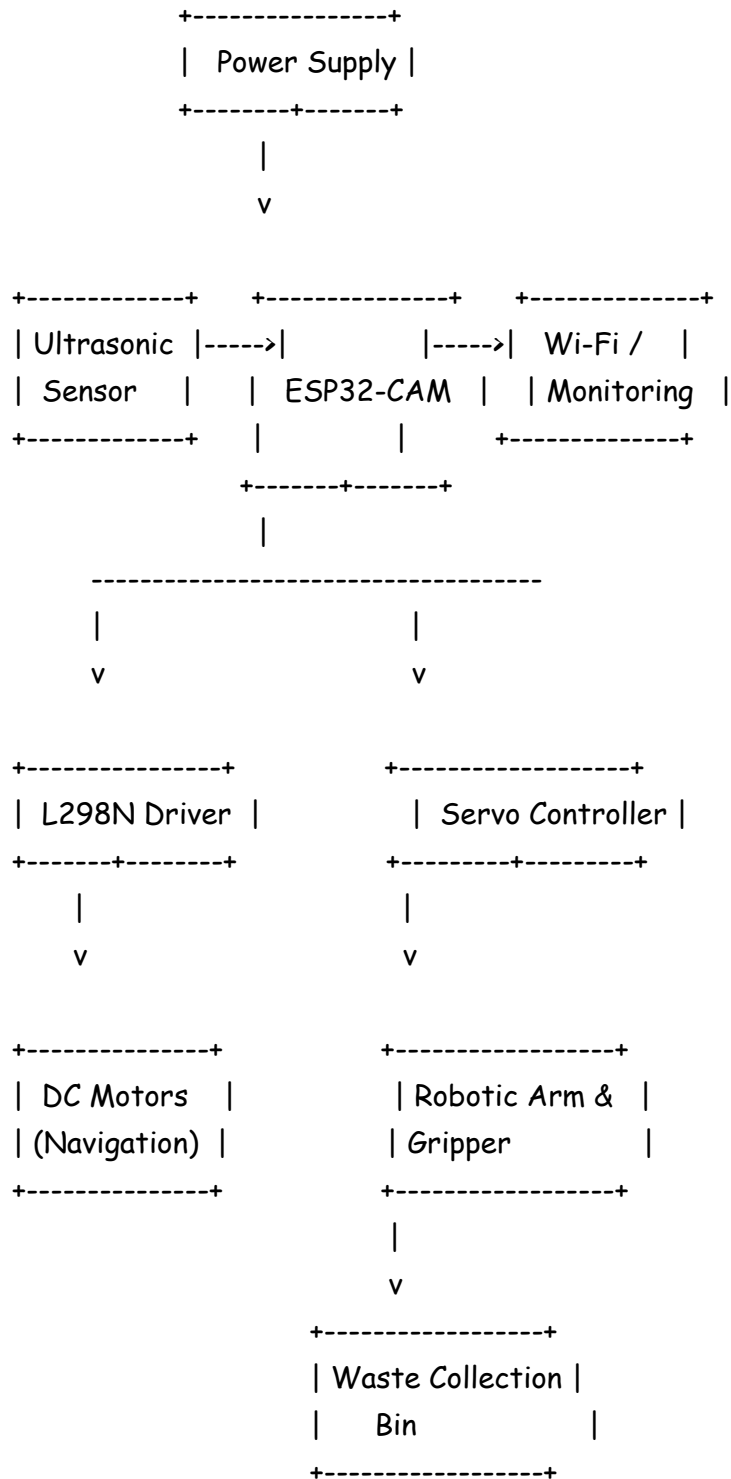
This system reduces manual labor, improves city cleanliness, and promotes efficient waste segregation.

2. Components List

Component	Purpose
ESP32-CAM	Main controller and vision processing
Ultrasonic Sensor(HC-SR04)	Obstacle detection
IR Sensor	Garbage detection assistance
L298N Motor Driver	Controls wheel motors
DC Motors	Robot movement
Servo Motor	Robotic arm movement
Servo Gripper	Pick-and-place operation
Battery Pack	Power source
Wi-Fi Module (Built into ESP32)	Data communication
Waste Storage Bins	Store collected garbage
Chassis	Robot body

3. Block Diagram

Figure 1: Block Diagram of Garbage Collection Robot



4. Circuit Diagram (ESP32-CAM Based)

Figure 2: Circuit Diagram of Garbage Collection Robot

```
      +-----+
      | ESP32-CAM |
      +-----+
```

Ultrasonic Sensor

VCC -----> 5V

GND -----> GND

TRIG -----> GPIO13

ECHO -----> GPIO12

L298N Motor Driver

ENA -----> GPIO14

IN1 -----> GPIO15

IN2 -----> GPIO2

IN3 -----> GPIO4

IN4 -----> GPIO16

ENB -----> GPIO0

Servo Motor (Arm)

Signal -----> GPIO3

Servo Motor (Gripper)

Signal -----> GPIO1

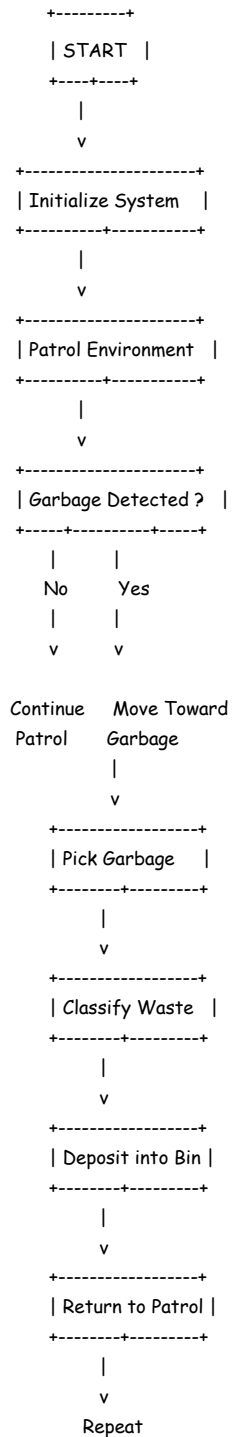
Power Supply

+5V -----> ESP32-CAM, Sensors

GND -----> Common Ground

5. Flowchart

Figure 3: Operational Flow of Garbage Collection Robot



6. Pseudocode

START

Initialize ESP32-CAM

Initialize Ultrasonic Sensor

Initialize Servo Motors

Initialize Motor Driver

Initialize Wi-Fi

WHILE Robot is Active

 Patrol Area

 IF Garbage Detected THEN

 Stop Robot

 Move Toward Garbage

 Position Arm

 Open Gripper

 Pick Garbage

 Close Gripper

 Classify Waste

 IF Recyclable THEN

 Move To Recyclable Bin

 ELSE

 Move To General Bin

 ENDIF

```
    Release Garbage
    Return Arm To Home Position
ENDIF
```

```
IF Obstacle Detected THEN
    Avoid Obstacle
ENDIF
ENDWHILE
END
```

7. Case Study / Iteration

Case 1: Plastic Bottle Found

Situation

The camera identifies a plastic bottle on the roadside.

Code Affected

```
if(object_type == "plastic")
{
    target_bin = RECYCLABLE;
}
```

Action

- Move to bottle
- Pick bottle
- Place in recyclable bin

Case 2: Food Waste Found

Situation

A banana peel is detected.

Code Affected

```
if(object_type == "organic")
{
    target_bin = GENERAL_WASTE;
}
```

Action

- Pick waste
- Place in general waste compartment

Case 3: Obstacle Encountered

Situation: A pedestrian walks in front of the robot.

Code Affected

```
if(distance < 30)
{
    stop();
    change_path();
}
```

Action

- Stop robot
- Recalculate route
- Continue operation

Case 4: Collection Bin Full

Situation

Storage compartment reaches capacity.

Code Affected

```
if(bin_level >= 100)
{
    send_alert();
    return_to_station();
}
```

Action

- Send notification
- Return to disposal station

Earthquake Monitoring and Evacuation Robot

Scenario

Design an Earthquake Monitoring and Evacuation Robot for disaster-prone buildings such as hospitals, shopping malls, and office complexes. The robot should continuously monitor seismic activity, detect earthquakes, assess environmental hazards (gas leaks, fire, blocked exits), and guide people toward the safest evacuation route. The robot should also send emergency alerts to the disaster management center through Wi-Fi.

1. Description

The Earthquake Monitoring and Evacuation Robot is an intelligent disaster-response system designed to reduce casualties during earthquakes. The robot continuously monitors vibrations using seismic sensors and accelerometers. Under normal conditions, it patrols the building and collects environmental data.

When abnormal ground vibrations are detected, the robot switches to emergency mode. It sounds alarms, identifies safe evacuation routes, and guides occupants using voice instructions, displays, and LED indicators. The robot also checks for secondary hazards such as gas leaks, fire outbreaks, and blocked exits. If a route becomes unsafe, the robot dynamically calculates an alternative path.

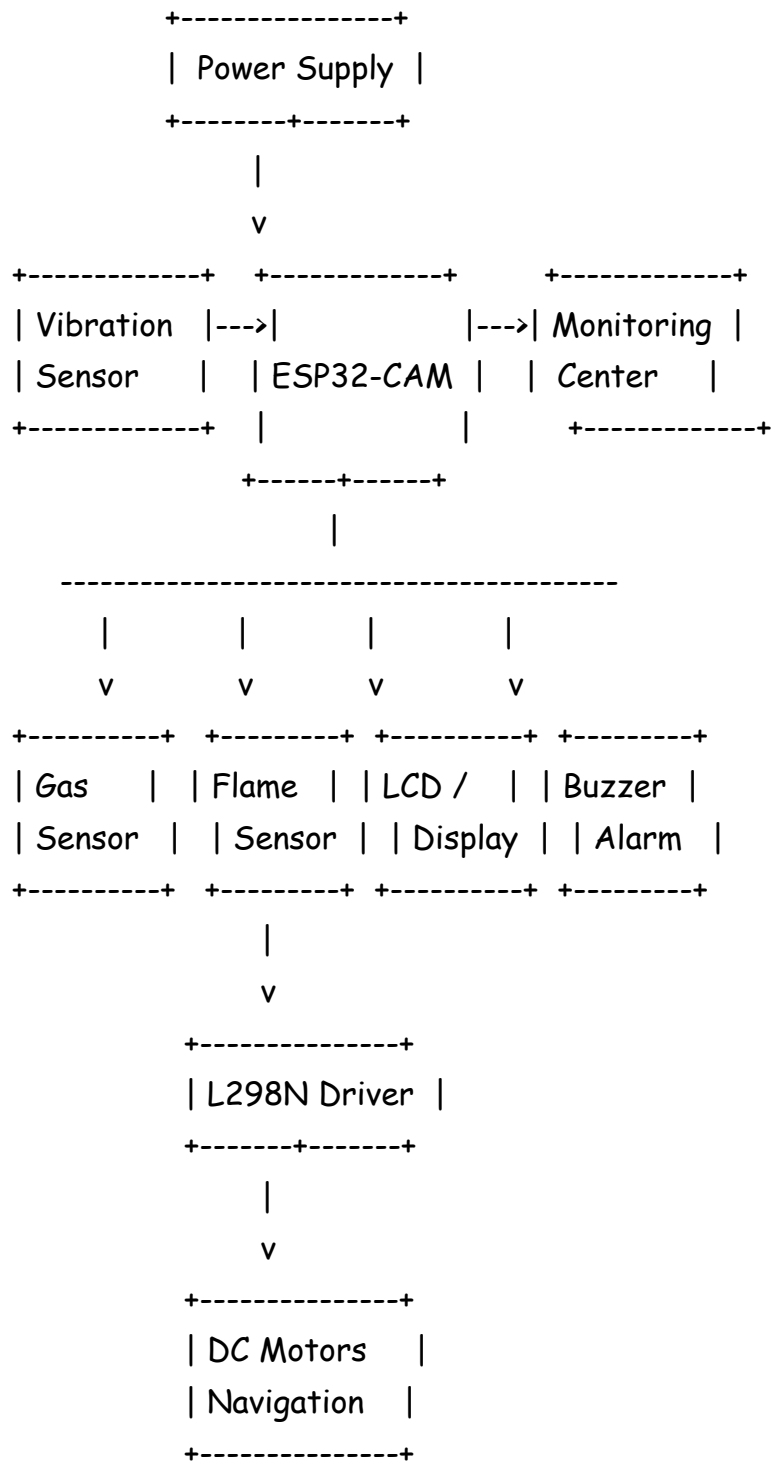
The collected information is transmitted to a central monitoring station, allowing rescue teams to make informed decisions.

2. Components List

Component	Purpose
ESP32-CAM	Main controller and vision processing
MPU6050 Accelerometer & Gyroscope	Earthquake/vibration monitoring
Vibration Sensor (SW-420)	Seismic activity detection
MQ-2 Gas Sensor	Gas leakage detection
Flame Sensor	Fire detection
Ultrasonic Sensor	Obstacle detection
L298N Motor Driver	Motor control
DC Motors	Robot movement
Buzzer	Emergency alarm
LCD/OLED Display	Emergency instructions
Battery Pack	Power source
Wi-Fi Module (Built-in ESP32)	Communication

3. Block Diagram

Figure 1: Block Diagram of Earthquake Monitoring and Evacuation Robot



4. Circuit Diagram (ESP32-CAM Based)

Figure 2: Circuit Diagram of Earthquake Monitoring and Evacuation Robot

```
+-----+  
|  ESP32-CAM  |  
+-----+
```

MPU6050

```
-----  
VCC -----> 3.3V  
GND -----> GND  
SDA -----> GPIO14  
SCL -----> GPIO15
```

Vibration Sensor

```
-----  
VCC -----> 3.3V  
GND -----> GND  
OUT -----> GPIO13
```

MQ-2 Gas Sensor

```
-----  
VCC -----> 5V  
GND -----> GND  
OUT -----> GPIO12
```

Flame Sensor

```
-----  
VCC -----> 5V  
GND -----> GND  
OUT -----> GPIO2
```

Buzzer

+ -----> GPIO4

- -----> GND

LCD Display (I2C)

SDA -----> GPIO14

SCL -----> GPIO15

L298N Driver

ENA -----> GPIO16

IN1 -----> GPIO0

IN2 -----> GPIO1

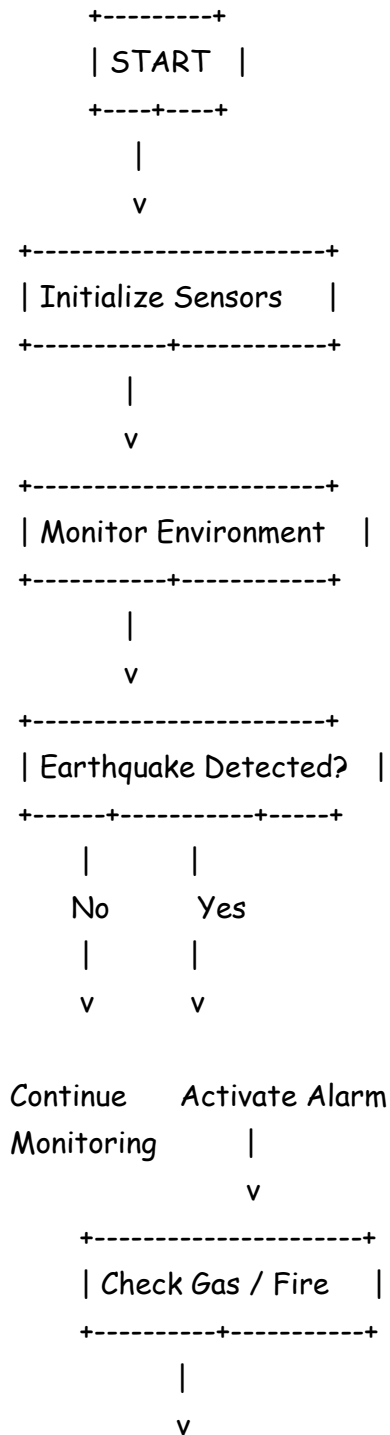
IN3 -----> GPIO3

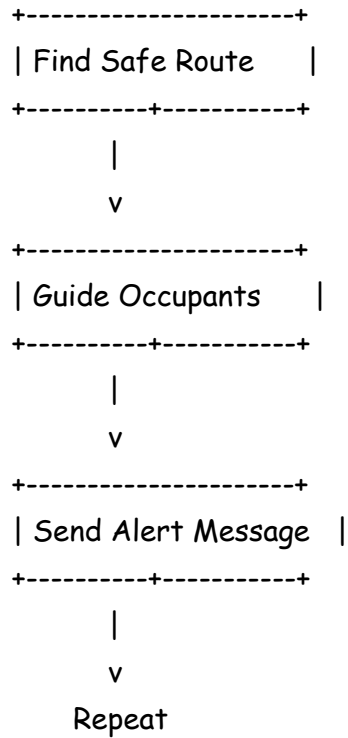
IN4 -----> GPIO4

DC Motors connected to L298N

5. Flowchart

Figure 3: Operational Flow of Earthquake Monitoring and Evacuation Robot





6. Pseudocode

START

Initialize MPU6050

Initialize Vibration Sensor

Initialize Gas Sensor

Initialize Flame Sensor

Initialize Motors

Initialize LCD

Initialize Wi-Fi

WHILE Robot is Active

 Read Seismic Data

 IF Earthquake Detected THEN

 Activate Alarm

```
    Check Gas Leakage
    Check Fire Hazard
    Determine Safe Route
    Display Exit Direction
    Navigate Toward Exit
    Send Emergency Alert
ENDIF

IF Obstacle Detected THEN
    Avoid Obstacle
ENDIF

ENDWHILE
END
```

7. Case Study / Iteration

Case 1: Minor Earthquake (Low Magnitude)

Situation

A mild tremor is detected that does not require full evacuation.

Code Affected

```
if(magnitude < 4.0)
{
    warning_mode = true;
}
```

Action

- Display warning message
- Continue monitoring
- No evacuation initiated

Case 2: Major Earthquake

Situation

A severe earthquake is detected.

Code Affected

```
if(magnitude >= 6.0)
{
    evacuation_mode = true;
}
```

Action

- Activate buzzer
- Begin evacuation guidance
- Send emergency alert

Case 3: Gas Leak Detected

Situation: A damaged pipeline releases gas after the earthquake.

Code Affected

```
if(gas_level > threshold) {
    unsafe_zone = true;
    select_alternate_route();
}
```

Action

- Mark area as dangerous
- Choose alternate evacuation route
- Notify monitoring center

Case 4: Fire Detected

Situation

Electrical short circuit causes a fire.

Code Affected

```
if(flame_detected)
{
    avoid_fire_area();
}
```

Action

- Avoid fire zone
- Redirect people through another exit

Case 5: Blocked Exit

Situation: Building debris blocks the primary evacuation route.

Code Affected

```
if(obstacle_distance < 20)
{
    recalculate_route();
}
```

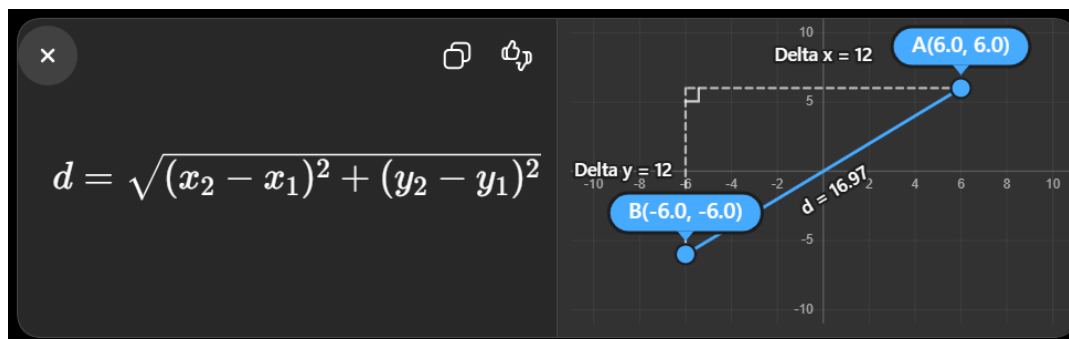
Action

- Detect blockage
- Compute alternate path
- Continue guidance

Vision System Challenges and Solutions

Challenge	Solution
Smoke reduces visibility	Use thermal/flame sensors
Low light after power outage	LED illumination
Dynamic obstacles (people)	Real-time obstacle avoidance
Debris blocking exits	Path replanning algorithm

Distance Calculation:



Turning Angle: $\theta = \tan^{-1}((y_2 - y_1)/(x_2 - x_1))$

Robotic Arm & Nursing

Robotic Arm / Robotic Manipulator

Scenario

Design a 6-DOF Robotic Arm for Fruit Picking and Packaging in Smart Agriculture. The robotic arm should identify ripe fruits using a camera, calculate their position, pick them without causing damage, and place them into packaging boxes. The system should be able to handle fruits of different sizes, heights, and ripeness levels.

1. Description

The Fruit Picking and Packaging Robotic Arm is an intelligent agricultural automation system designed to reduce manual labor and improve harvesting efficiency. The robot uses an ESP32-CAM to detect fruits and determine their ripeness through image processing. Once a ripe fruit is identified, the robotic arm calculates the required joint angles using **Inverse Kinematics (IK)** and moves its end-effector (gripper) toward the fruit.

The gripper gently picks the fruit and transfers it to a packaging area. The arm consists of six degrees of freedom (DOF), allowing movement similar to a human arm. Sensors provide feedback to ensure accurate positioning and safe operation.

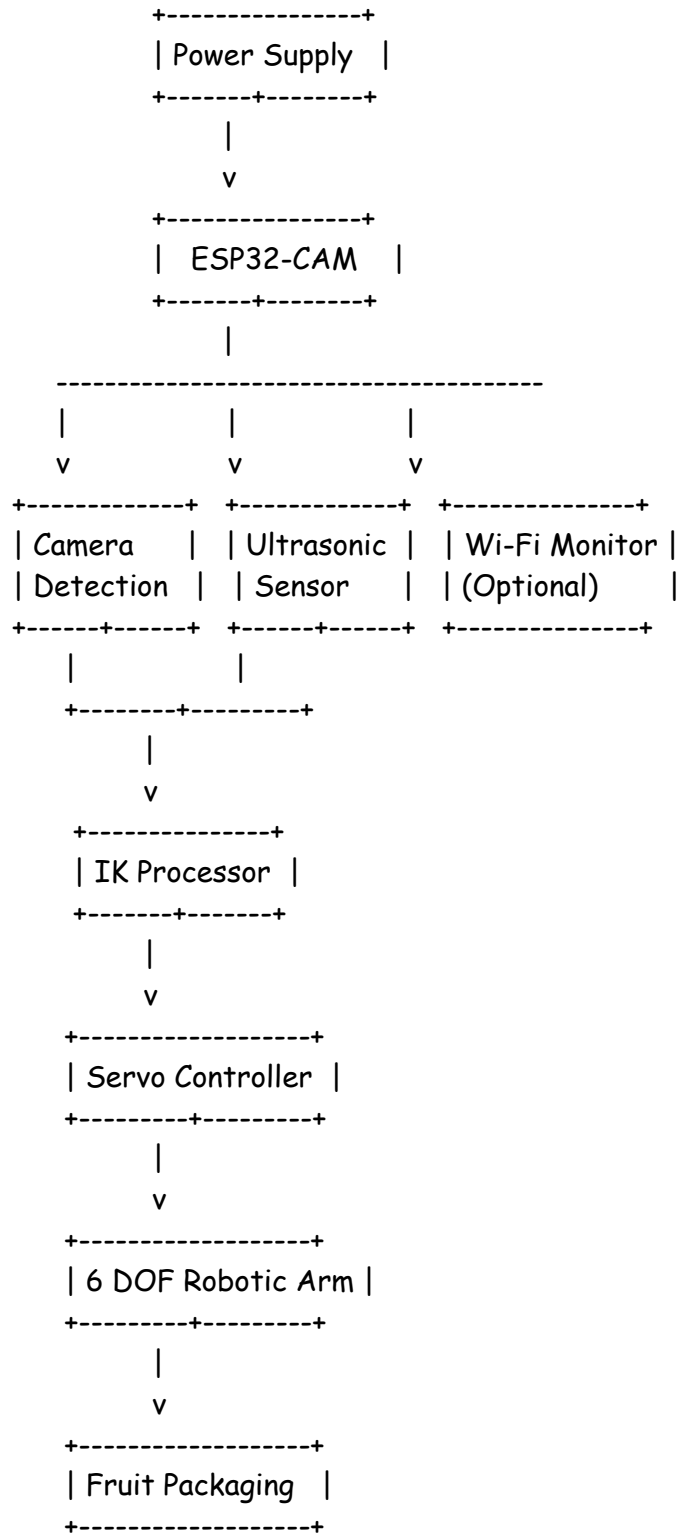
This system minimizes fruit damage, increases harvesting speed, and supports precision agriculture.

2. Components List

Component	Purpose
ESP32-CAM	Main controller and vision processing
6 Servo Motors	Joint actuation
Robotic Gripper	Fruit gripping
Ultrasonic Sensor	Distance measurement
Color/Camera Processing	Ripeness detection
Servo Driver(PCA9685)	Controls multiple servos
Power Supply	System power
Packaging Bin	Fruit collection
Robotic Arm Frame	Mechanical structure

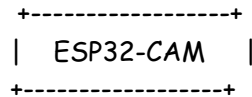
3. Block Diagram

Figure 1: Block Diagram of Fruit Picking Robotic Arm



4. Circuit Diagram (ESP32-CAM Based)

Figure 2: Circuit Diagram of Fruit Picking Robotic Arm



Ultrasonic Sensor

VCC -----> 5V
GND -----> GND
TRIG -----> GPIO13
ECHO -----> GPIO12

PCA9685 Servo Driver

SDA -----> GPIO14
SCL -----> GPIO15
VCC -----> 5V
GND -----> GND

Servo Connections

CH0 -> Base Servo
CH1 -> Shoulder Servo
CH2 -> Elbow Servo
CH3 -> Wrist Pitch
CH4 -> Wrist Roll
CH5 -> Gripper Servo

ESP32-CAM Camera

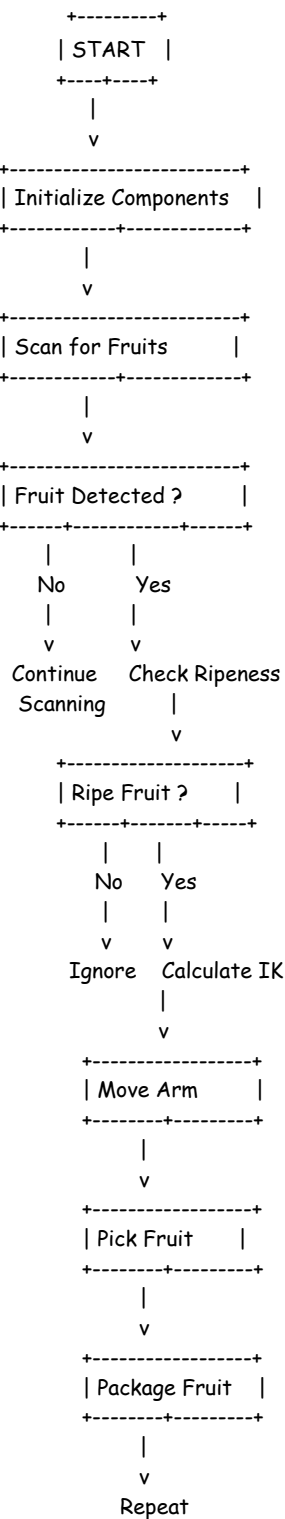
Built-in Camera Used
For Fruit Detection

Power Supply

5V -> PCA9685
5V -> ESP32-CAM
GND -> Common Ground

5. Flowchart

Figure 3: Operational Flow of Fruit Picking Robot



6. Pseudocode

START

Initialize Camera

Initialize Ultrasonic Sensor

Initialize Servo Controller

Initialize Robotic Arm

WHILE System Active

 Scan For Fruit

 IF Fruit Found THEN

 Check Ripeness

 IF Fruit Is Ripe THEN

 Calculate Fruit Position

 Apply Inverse Kinematics

 Move Arm To Fruit

 Open Gripper

 Pick Fruit

 Close Gripper

 Move To Packaging Box

 Release Fruit

 Return To Home Position

 ENDIF

 ENDIF

ENDWHILE

END

7. Case Study / Iterations

Case 1: Soft Fruit (Strawberry)

Problem: High gripping force damages the fruit.

Code Affected

```
grip_force = LOW;  
servo_speed = SLOW;
```

Action

- Reduce gripping force
- Slow arm movement

Case 2: Fruit on High Branch

Problem: Fruit is outside normal reach.

Code Affected

```
shoulder_angle += 20;  
elbow_angle += 15;
```

Action

- Increase arm extension
- Recalculate IK

Case 3: Unripe Fruit

Problem: Green fruit should not be harvested.

Code Affected

```
if(ripeness < 80) {  
    skip_fruit();  
}
```

Action

- Ignore fruit
- Continue searching

Case 4: Large Fruit

Problem: Fruit weight is higher than expected.

Code Affected

```
grip_force = HIGH;  
servo_speed = MEDIUM;
```

Action

- Increase gripping force
- Reduce sudden movements

Case 5: Obstacle (Tree Branch)

Problem: Branch blocks arm movement.

Code Affected

```
if(obstacle_detected) {  
    recalculate_path();  
}
```

Action

- Compute alternate trajectory
- Continue picking operation

8. Torque Analysis

$$\tau = F \times r$$

Where:

- τ = Torque
- F = Force due to fruit weight
- r = Distance from joint

Example

Fruit Weight = 2 N

Distance from Shoulder Joint = 0.25 m

$$\begin{aligned}\text{Torque} &= 2 \times 0.25 \\ &= 0.5 \text{ Nm}\end{aligned}$$

The shoulder motor must provide more than **0.5 Nm** torque.

Nursing Assistant Robot

Scenario

Design a Nursing Assistant Robot for a hospital that can deliver medicines, monitor patient vital signs, assist doctors and nurses, and provide emergency alerts when abnormal conditions are detected. The robot should autonomously navigate hospital corridors, avoid obstacles, communicate with hospital staff, and maintain patient records.

1. Description

The Nursing Assistant Robot is a healthcare support robot designed to reduce the workload of nurses and improve patient care. The robot moves autonomously throughout hospital wards, delivering medicines and essential supplies to patients according to a predefined schedule.

The robot continuously monitors vital signs such as body temperature, heart rate, and oxygen saturation (SpO_2). If any abnormal condition is detected, the robot immediately notifies nurses and doctors. Using its camera and sensors, the robot can identify patient rooms, avoid obstacles, and communicate with patients through voice or display messages.

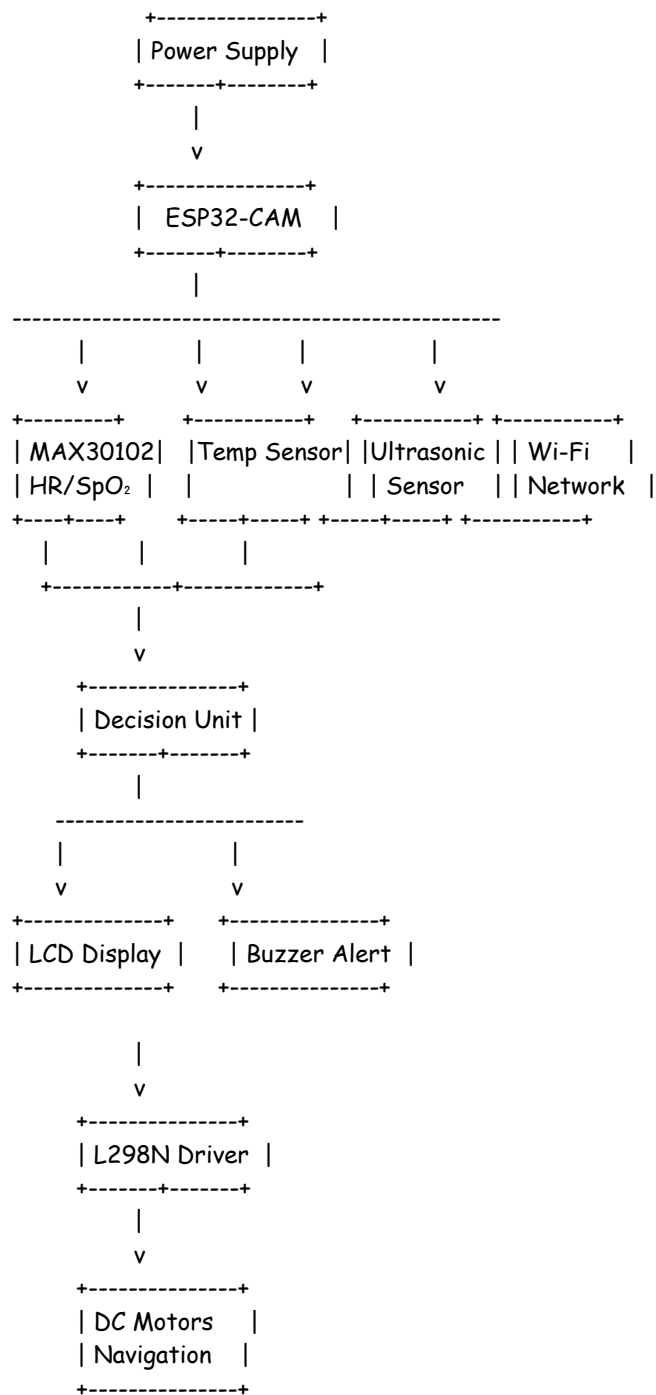
The system improves efficiency, reduces human error, and enables faster emergency response.

2. Components List

Component	Purpose
ESP32-CAM	Main controller and vision processing
MAX30102 Sensor	Heart Rate & SpO ₂ monitoring
LM35 / DS18B20	Body temperature monitoring
Ultrasonic Sensor	Obstacle detection
L298N Motor Driver	Controls movement
DC Motors	Robot locomotion
LCD/OLED Display	Display instructions
Buzzer	Emergency alerts
Servo Motor	Medicine compartment control
Battery Pack	Power supply
Wi-Fi Module (ESP32 Built-in)	Communication with nurses

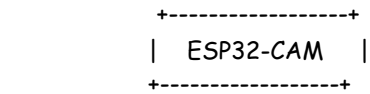
3. Block Diagram

Figure 1: Block Diagram of Nursing Assistant Robot



4. Circuit Diagram (ESP32-CAM Based)

Figure 2: Circuit Diagram of Nursing Assistant Robot



MAX30102

VCC -----> 3.3V
GND -----> GND
SDA -----> GPIO14
SCL -----> GPIO15

Temperature Sensor

VCC -----> 3.3V
GND -----> GND
OUT -----> GPIO13

Ultrasonic Sensor

VCC -----> 5V
GND -----> GND
TRIG -----> GPIO12
ECHO -----> GPIO2

Buzzer

+ -----> GPIO4
- -----> GND

Servo Motor

Signal -----> GPIO16

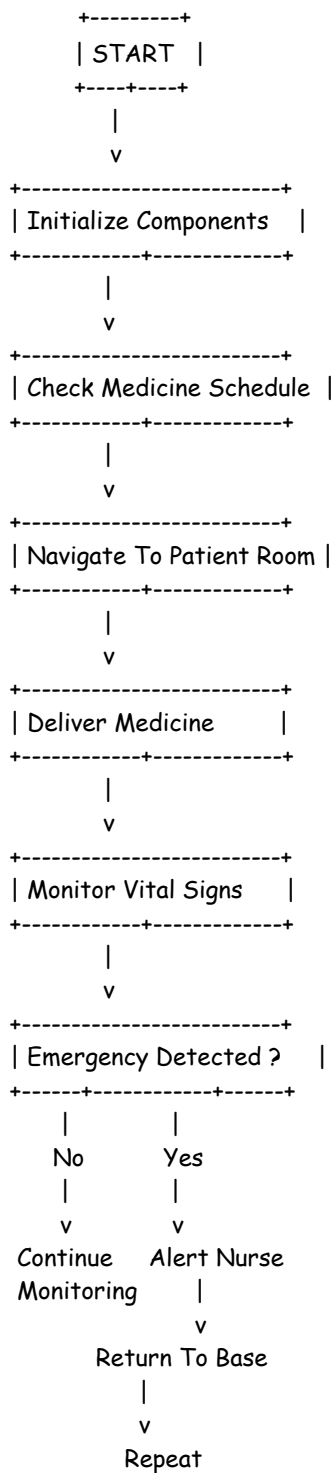
L298N Driver

ENA -----> GPIO0
IN1 -----> GPIO1
IN2 -----> GPIO3
IN3 -----> GPIO12
IN4 -----> GPIO13

DC Motors connected to L298N

5. Flowchart

Figure 3: Operational Flow of Nursing Assistant Robot



6. Pseudocode

START

Initialize Camera

Initialize MAX30102

Initialize Temperature Sensor

Initialize Motors

Initialize Wi-Fi

WHILE Robot Active

 Check Medicine Schedule

 Navigate To Patient Room

 Deliver Medicine

 Measure Heart Rate

 Measure SpO2

 Measure Temperature

 IF Vital Signs Abnormal THEN

 Activate Alarm

 Send Alert To Nurse

 ENDIF

 Return To Charging Station

ENDWHILE

END

7. Case Study / Iterations

Case 1: Scheduled Medicine Delivery

Situation: Patient in Room 205 requires medicine at 10:00 AM.

Code Affected

```
if(current_time == medicine_time)
{
    deliver_medicine();
}
```

Action

- Navigate to Room 205
- Open medicine compartment
- Notify patient

Case 2: High Fever Detected

Situation: Patient temperature rises above 39°C.

Code Affected

```
if(temperature > 39)
{
    send_alert();
}
```

Action

- Alert nurse station
- Display emergency message

Case 3: Low Oxygen Saturation

Situation: SpO₂ falls below 90%.

Code Affected

```
if(spo2 < 90)
{
    emergency_mode = true;
}
```

Action

- Trigger buzzer
- Notify medical staff immediately

Case 4: Corridor Obstruction

Situation

A wheelchair blocks the path.

Code Affected

```
if(distance < 30)
{
    stop();
    reroute();
}
```

Action

- Stop movement
- Calculate alternate route

Case 5: Patient Calls for Help

Situation: Emergency button is pressed by patient.

Code Affected

```
if(help_button_pressed)
{
    go_to_patient();
}
```

Action

- Navigate to patient's location
- Establish communication
- Alert nurse station

Firefighting

Firefighting Robot

Scenario

Design a Firefighting Robot for industrial buildings, warehouses, and office complexes. The robot should continuously patrol designated areas, detect fire and smoke, navigate autonomously toward the source, avoid obstacles, extinguish the fire using a water pump, and send emergency alerts to a control center.

1. Description

The Firefighting Robot is an autonomous emergency response robot designed to reduce fire-related damage and improve safety in hazardous environments. The robot continuously monitors its surroundings using flame sensors, smoke sensors, and a camera. When a fire is detected, it determines the location of the flame and moves toward it while avoiding obstacles.

Once it reaches a safe extinguishing distance, the robot activates a water pump and sprays water until the fire is extinguished. If the fire spreads or becomes inaccessible, the robot alerts firefighters and transmits location information through Wi-Fi.

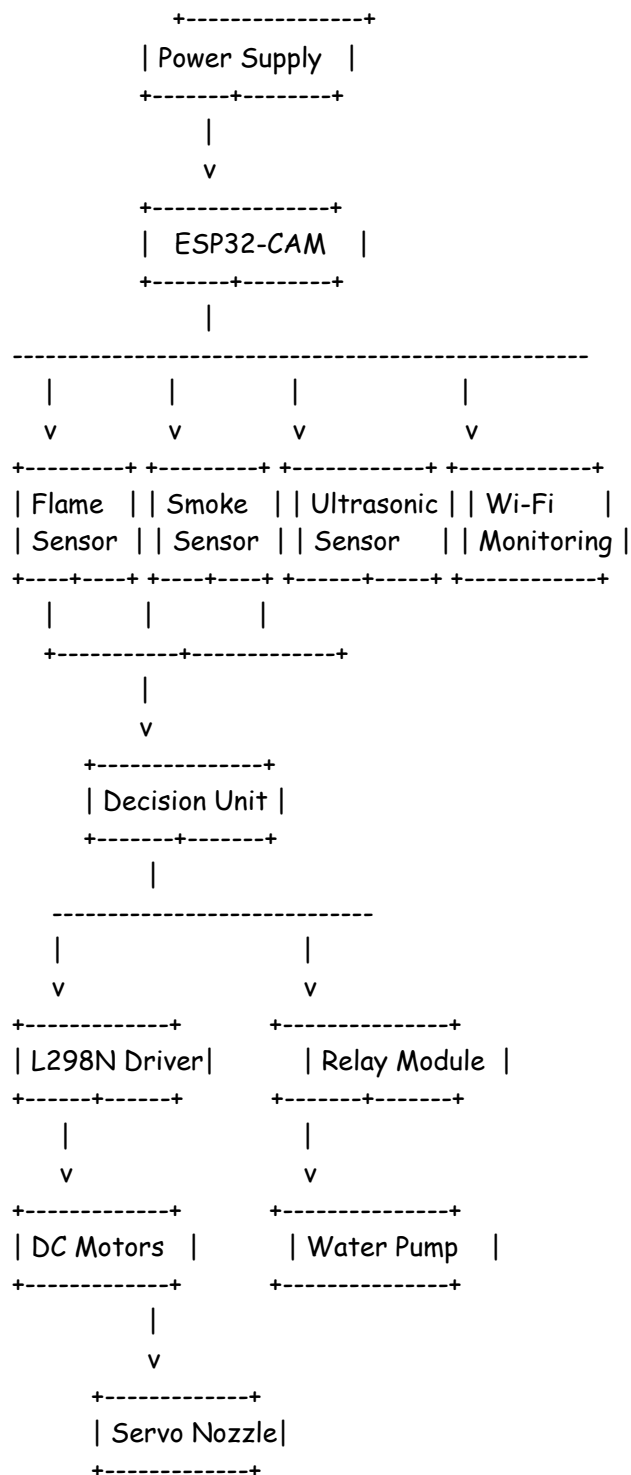
This robot is particularly useful in environments where human firefighters may face significant risks, such as chemical plants, factories, and storage facilities.

2. Components List

Component	Purpose
ESP32-CAM	Main controller and vision processing
Flame Sensor	Fire detection
MQ-2 Smoke Sensor	Smoke detection
Ultrasonic Sensor	Obstacle detection
L298N Motor Driver	Controls wheel motors
DC Motors	Robot movement
Relay Module	Controls water pump
Water Pump	Fire extinguishing
Servo Motor	Water nozzle rotation
Buzzer	Emergency alarm
Battery Pack	Power source
Wi-Fi Module (ESP32 Built-in)	Communication

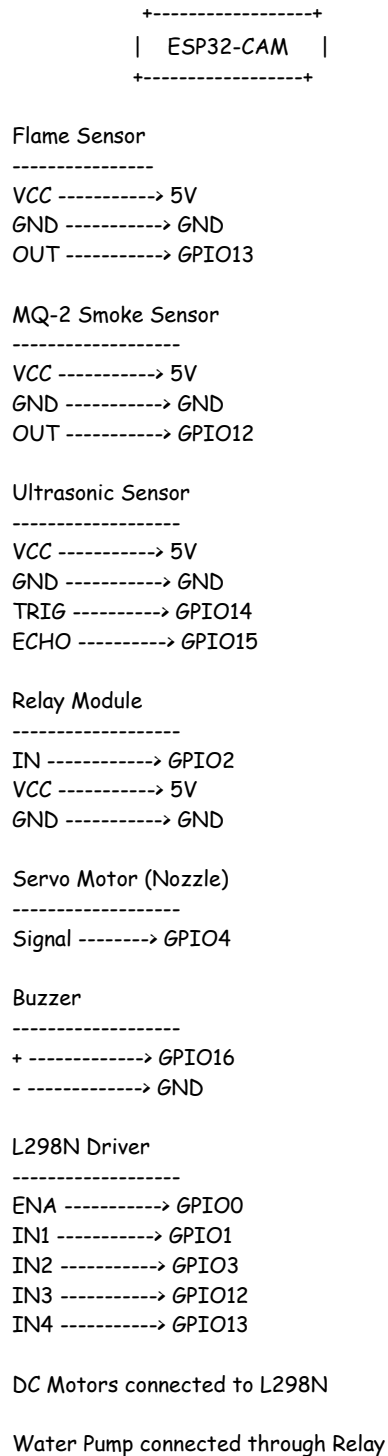
3. Block Diagram

Figure 1: Block Diagram of Firefighting Robot



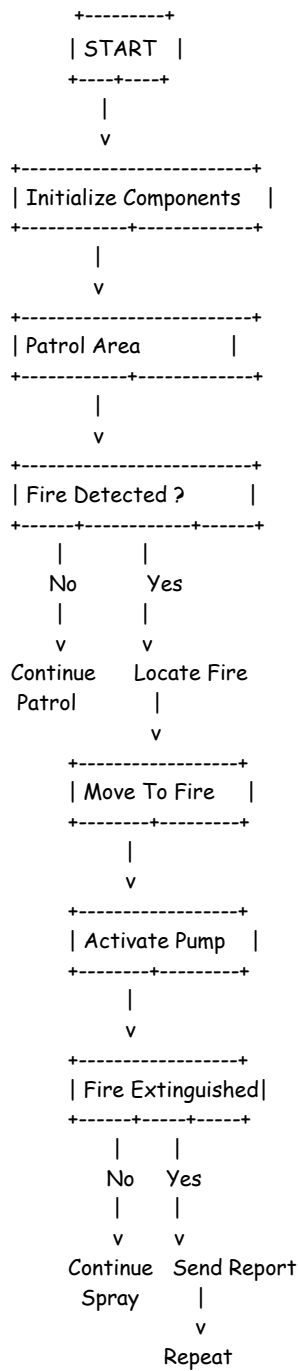
4. Circuit Diagram (ESP32-CAM Based)

Figure 2: Circuit Diagram of Firefighting Robot



5. Flowchart

Figure 3: Operational Flow of Firefighting Robot



6. Pseudocode

START

Initialize Flame Sensor
Initialize Smoke Sensor
Initialize Ultrasonic Sensor
Initialize Motors
Initialize Relay
Initialize Pump

WHILE Robot Active

 Patrol Area

 IF Fire Detected THEN

 Stop Patrol

 Locate Fire Source

 Move Toward Fire

 Activate Water Pump

 WHILE Fire Exists

 Continue Spraying

 ENDWHILE

 Turn Off Pump

 Send Status Report

 ENDIF

 IF Obstacle Detected THEN

 Avoid Obstacle

 ENDIF

ENDWHILE

END

7. Case Study / Iterations

Case 1: Small Fire

Situation: A small paper fire occurs in a room.

Code Affected

```
if(flame_level < 30)
{
    pump_speed = LOW;
}
```

Action

- Move toward fire
- Spray water at low flow rate
- Extinguish fire

Case 2: Large Fire

Situation: A large fire spreads across multiple shelves.

Code Affected

```
if(flame_level > 70)
{
    pump_speed = HIGH;
}
```

Action

- Increase water pressure
- Continue spraying longer

Case 3: Smoke Detected Before Flame

Situation: Smoke appears before visible fire.

Code Affected

```
if(smoke_level > threshold)
{
    investigate_area();
}
```

Action

- Move toward smoke source
- Search for fire
- Alert control center

Case 4: Obstacle Between Robot and Fire

Situation: A fallen object blocks the route.

Code Affected

```
if(distance < 20)
{
    stop();
    recalculate_path();
}
```

Action

- Avoid obstacle
- Find alternate route
- Continue mission

Case 5: Water Tank Empty

Situation: The water reservoir becomes empty during operation.

Code Affected

```
if(water_level == 0)
{
    return_to_base();
    send_alert();
}
```

Action

- Stop firefighting operation
- Notify control center
- Return for refill